



FINAL PROJECT DOCUMENT

SyllabusOS

Knows what you don't know. Teaches it without doing it for you.

Team Crazy Coders · Horizon by Hoolow — Round 1 (Online MVP) · Theme: AI with Education

Categories: Education · Student Life · Productivity · 29 September 2026

At a glance. SyllabusOS turns a student's syllabus, notes and previous-year papers into an adaptive learning loop. It diagnoses concept-level weaknesses, uses an explainable engine to decide what to study next, and teaches through a grounded Socratic tutor that refuses to just hand over answers. Every reply updates mastery and re-plans the day.

1 Problem Statement

Students are not short of content. They are short of **decisions** and **real understanding**. Four failure modes show up again and again:

- **No prioritisation.** A syllabus lists 30+ topics with no order, no weightage and no link to the exam date or the student's free time. Students start at Unit 1 and run out of time.
- **No self-knowledge.** Students cannot tell what they know from what they have only read. The gap shows up in the exam hall.
- **Static plans die on day two.** Miss one session and a fixed timetable turns into a backlog.
- **AI makes it worse.** General chatbots are answer engines: paste the question, copy the answer, feel competent, fail the offline exam. Chatbot "study modes" help, but they don't know your syllabus, exam date, weak topics or past mistakes.

Core insight: the gap is not information, it is **execution** — knowing what to do next, and actually learning it instead of copying it.

Who it is for: university students preparing for semester exams, starting with Indian universities, where syllabi are organised in units and preparation is driven by previous-year questions (PYQs).

2 Solution: One Closed Loop

SyllabusOS is not another chatbot; it is the decision layer between a student and everything they need to learn. The product is a single loop, and every AI interaction must produce a visible change in the student's learning state.

Set-up takes a few minutes. The student sets a goal (subject, exam date, minutes per day), uploads a syllabus that becomes an editable concept graph, optionally adds PYQs for real weightage, and takes a short diagnostic. From then on one button — **Study Now** — returns one concept, a duration and a reason. The session that follows is Socratic; every reply is graded into evidence, mastery updates, and the plan re-ranks (Figure 1).

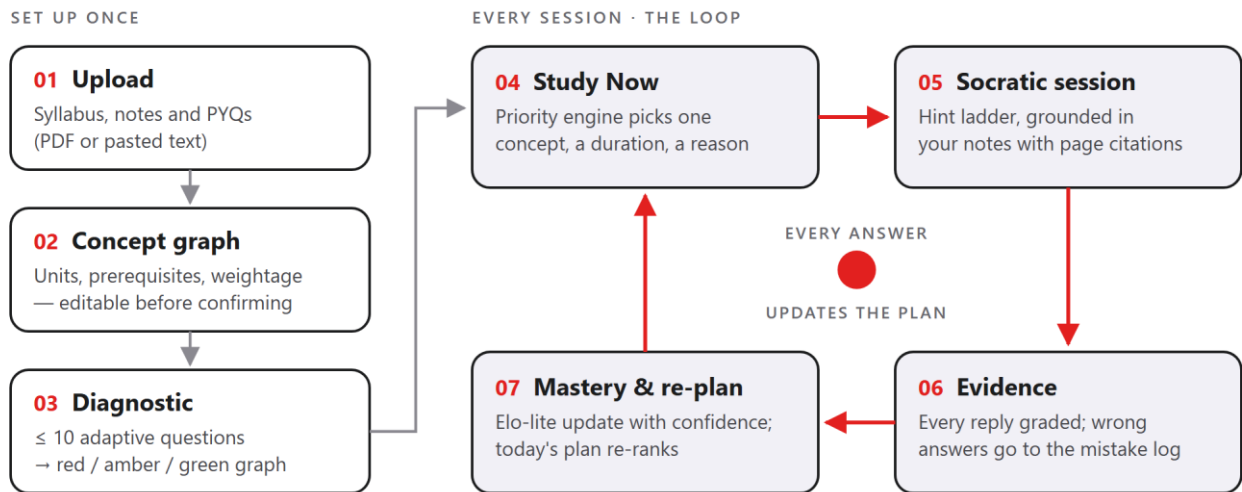


Figure 1. Three set-up steps, then a loop that runs on every study session.

3 What Makes It Different

Differentiator	How it works	Why it matters
The decision engine is code, not vibes	The LLM never decides what you study. A deterministic, unit-tested priority engine does, from mastery, exam weightage, prerequisite unlocks, mistakes and time left.	Every recommendation carries a reason built from the real score components — no hallucinated advice, fully explainable.
Socratic by default, every exchange measured	A hint ladder: probe → hint → stronger hint → worked step → check. Each reply is graded correct / partial / wrong with a misconception label.	Learning is the assessment. Students cannot shortcut to the answer, and the system learns where they are stuck.
Weightage from real previous-year papers	Each PYQ is mapped to a concept to compute the actual share of marks per topic for the student's own university. Without PYQs, an LLM estimate is used and labelled "estimated".	Matches how students actually prepare and spends limited time on what the exam rewards.

Example of a recommendation the student sees: *"Conflict Serializability · 25 min — High exam weightage (18% of PYQ marks) and it unlocks Recoverability."*

4 Features and Scope

MVP (P0) — must work end to end

#	Feature	Acceptance criteria
1	Goal setup	Subject, exam date and minutes per day on one screen.
2	Syllabus → concept graph	PDF or pasted text → 20–40 concepts in units with prerequisite edges in under 30 s; rename, delete or add concepts before confirming.
3	PYQ weightage (optional)	Every question mapped to a concept → weightage bar per topic; "estimated" badge when no PYQs are given.
4	Adaptive diagnostic	≤ 10 MCQ / numeric questions, high-weightage and prerequisite-root concepts first; the graph recolours red / amber / green.
5	Study Now + today's plan	One concept + duration + reason; the day's plan fills the available minutes and re-plans instantly on any change.

#	Feature	Acceptance criteria
6	Socratic learn session	Grounded in the student's notes with page citations; hint ladder; 2–3 check questions; mastery before → after.
7	Mistake log	Every wrong answer stored with its concept and misconception; recurring misconceptions raise that concept's priority.
8	Dashboard	Coloured concept graph, today's plan, weakest three topics and mastery trend.
9	Guest demo login	"Try demo" opens a seeded account — no sign-up needed.

Next (P1) and explicitly out of scope

- **P1:** Revision Mode ("I have 90 minutes before the exam"), spaced-repetition decay, Explain My Mistake (why wrong → correct reasoning → similar retry question), Hinglish explanations and Triage Mode. The engine logic for revision, decay and triage is already implemented.
- **Cut from Round 1:** real-time audio rooms, a keystroke-tracking IDE, live multiplayer, an "exam readiness %" score, native mobile apps, payments, notifications and social features. Each adds fragile real-time infrastructure or makes a claim we cannot back with data.

5 System Architecture

One TypeScript codebase, deployed as one Next.js app with one Postgres database and one LLM provider — every extra service is another thing that can break during a live demo.

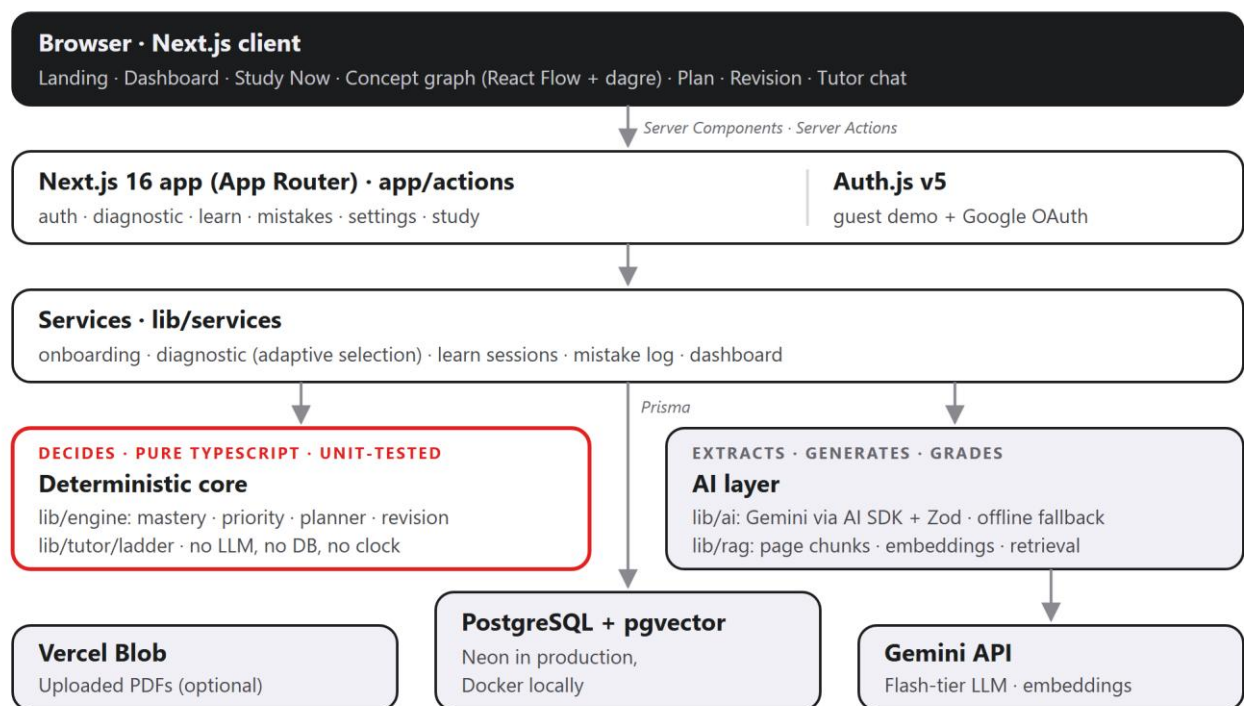


Figure 2. Request flow. Decisions come from the pure core; the AI layer only extracts, generates and grades.

- **Server-first.** Pages are React Server Components; user actions call Server Actions (app/actions), which call the services layer. The only HTTP route is the Auth.js handler.
- **Pure core.** lib/engine and lib/tutor/ladder.ts use no database, no LLM and no clock (now is passed in), so the same inputs always produce the same plan. The engine is unit-testable and every page is reload-safe.
- **Graceful degradation.** Every AI function has an offline fallback and file storage is optional, so the full loop still runs without an API key (Section 7).

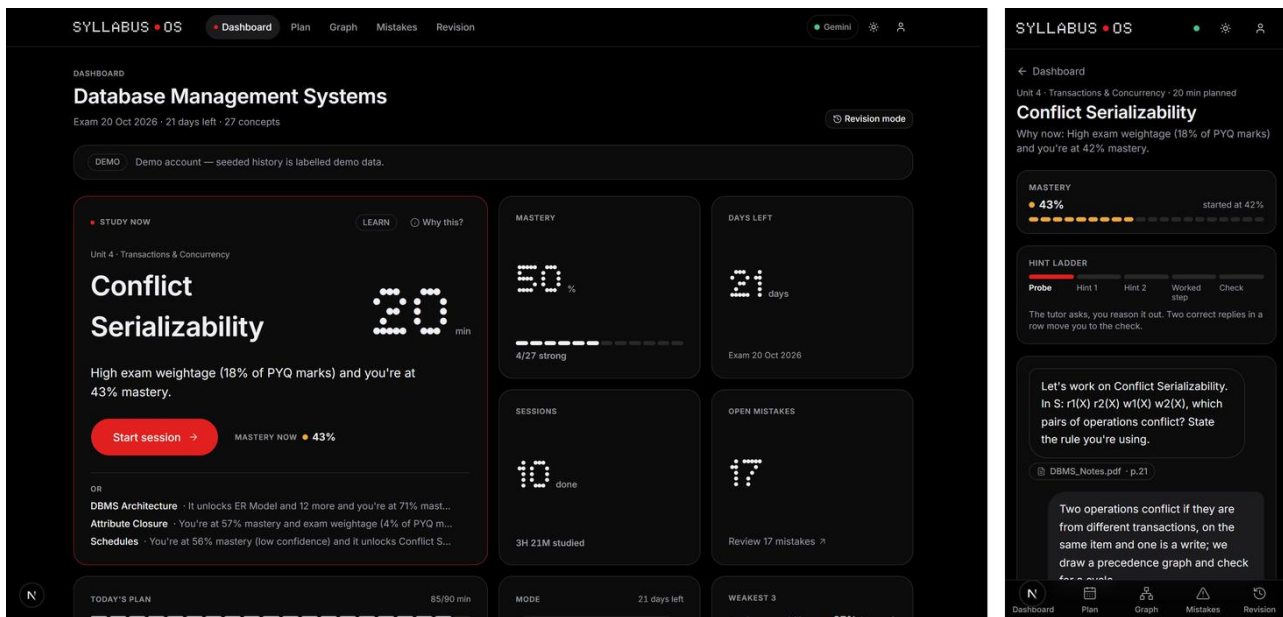


Figure 3. The running app: dashboard with the Study Now pick (desktop) and a Socratic session with a page citation (phone).

6 Core Engines

6.1 Mastery model (Elo-lite, per concept)

Each concept stores a skill value θ (in logits), shown to the student as mastery = $\sigma(\theta)$, and a weighted evidence count n . For a question of difficulty d , the expected chance of a correct answer is p ; after each answer θ moves towards the observed outcome (1, 0.5 or 0), and the step size K shrinks as evidence grows.

$$\begin{aligned} p &= \sigma(\theta - d) \\ \theta &\leftarrow \text{clamp}(\theta + K(n) \cdot w \cdot (\text{outcome} - p), -4, 4) \\ K(n) &= \max(0.25, 1.2 / (1 + n/4)) \end{aligned}$$

- **Evidence weight w :** quiz, diagnostic and check answers 1.0; Socratic replies 0.5 (LLM-graded, so trusted less). A unit-level diagnostic answer gives each child concept a low-confidence prior of $0.3 / (1 + n)$ — this is how ten questions honestly cover 30+ concepts.
- **Confidence:** none, low ($n < 2$), medium ($n < 5$) or high. Low-confidence concepts are drawn dashed; a one-question guess is never presented as fact.
- **Decay:** retention = $0.5^{(\text{days} / \text{half-life})}$, with half-life = $3 + 1.5 \cdot n$ days. Only mastery above 50% fades, by at most 60% of that margin — forgetting pulls a student back to “unsure”; it never invents evidence of weakness.

6.2 Priority engine (deterministic)

$$\begin{aligned} \text{value}(c) &= 0.30 \cdot \text{Weightage} + 0.30 \cdot \text{Gap} + 0.20 \cdot \text{Unlock} + 0.10 \cdot \text{MistakeRate} + 0.10 \cdot \text{Decay} \\ \text{priority}(c) &= \text{value}(c) / \sqrt{(\max(\text{minutesNeeded}, 10) / 10)} \end{aligned}$$

- All inputs are normalised to 0–1: Gap = $1 - \text{mastery}$; Unlock = concepts this one unblocks; MistakeRate = $(\text{recent mistakes} + 1.5 \times \text{recurring misconceptions}) / 4$, capped at 1. Dividing by time left means a quick high-value concept beats a slow one of equal value.
- **Prerequisite gating:** if a prerequisite is below 50% mastery, the engine follows the chain and recommends the weakest blocker instead, e.g. “Blocked by Functional Dependency (32% mastery) — it’s a prerequisite for Normalization.”
- **Reasons are templates, not generations:** the reason is built from the top two weighted components. No LLM is involved.
- **The deadline sets the mode** rather than multiplying every score (which would not change the ranking): **Learn** normally; **Revision** when 3 or fewer days remain; **Triage** when minutes needed exceed minutes

available — the lowest-weightage topics are dropped with a stated reason, and a prerequisite of a kept topic is never dropped.

6.3 Planner and Revision Mode

The planner greedily fills each day from the priority queue (prerequisites first) in 10–45 minute learn blocks. On days of 45 minutes or more it reserves a 15-minute practice block for the most mistake-prone concept and a 10-minute revision block for the most faded one. Skipped days get no blocks and their work flows forward across a 14-day horizon. The plan is rebuilt whenever a session ends, a day is skipped, minutes per day change or mastery changes. Revision Mode answers “I have N minutes” with blocks ranked by $(1 - \text{mastery}) \times \text{weightage} \times (0.5 + 0.5 \cdot \text{decay})$.

6.4 Adaptive diagnostic

Candidate concepts are scored $0.6 \times \text{weightage} + 0.4 \times \text{transitive dependents}$, so high-weightage prerequisite roots come first. After a wrong answer the diagnostic probes down to a prerequisite; after a correct answer it probes up to a high-value dependent; a partial answer spends the next question on an uncovered unit. It never asks about the same concept twice and stops at ten questions. State is rebuilt from stored attempts, so the same answers always lead to the same next question.

6.5 Socratic tutor and hint ladder

The LLM grades each reply and writes the message; a deterministic state machine decides where the session goes next.

Student signal	Ladder response
Correct at the probe stage	Stay for a deeper question; two correct in a row → check questions.
Correct with help	After a hint or worked step, move straight to check questions.
Wrong or partial	After two failed replies, move one rung up: probe → hint → stronger hint → worked step.
“Give me a hint”	Move one rung up immediately.
“Just tell me the answer”	Reply with a guiding question. The final answer never appears before the worked step.

Every tutor turn is schema-validated JSON: message, stage, evaluation, misconception, concept, page citations and a notInNotes flag that shows a visible “not found in your notes” label. Objective answers are never graded by the LLM: MCQs are exact, and numeric answers get full credit within 1% and half credit within 5%, with units, fractions and Indian digit grouping (“1,00,000”) parsed correctly. Wrong answers go to the mistake log with their misconception label, and recurring labels raise that concept’s priority.

7 AI Layer and Grounding

Gemini is reached through the Vercel AI SDK v7 (structured output validated with Zod schemas). The implementation:

- **Model access and resilience:** model IDs live in environment variables. When the primary model returns 503 (overloaded) or 429, the call moves down a comma-separated fallback list, and the overloaded model is skipped for two minutes (a circuit breaker). Tutor attempts time out after 12 s; if every model fails the offline tutor answers instantly. Measured on 29 Sep 2026: gemini-3.6-flash 4.6 s and gemini-3.1-flash-lite 3.7 s per structured call.
- **Bank first:** hand-verified bank questions are used whenever they cover a concept; Gemini writes questions only for concepts the bank doesn’t cover.

- **Structured output everywhere:** syllabus extraction, PYQ mapping, question generation, the question verify pass, tutor turns and mistake explanations are each validated against a shared Zod schema, with one retry on invalid output. The same schemas type the server and the UI.
- **Verify pass:** every generated question is checked by a second model call; questions whose answer key is disputed are dropped.
- **Clean graphs:** extracted prerequisite edges are de-duplicated and any edge that would close a cycle is removed, so the concept graph is always a DAG; weightage is normalised to 100%.
- **Grounding (RAG):** syllabus and PYQ PDFs go to Gemini directly (it handles scans and tables). Notes are split per page (unpdf) into ~800-token chunks that never cross a page boundary, embedded (768 dimensions) into pgvector and retrieved top-k by concept — so every citation points to a real page.
- **Offline mode:** without an API key the app still runs: a heuristic syllabus parser (understands “UNIT I”, “Module 3”, hours and boilerplate sections), a built-in DBMS question bank, keyword retrieval and a rule-based tutor. An on-screen chip shows which mode is active.

8 Data Model

PostgreSQL with pgvector, accessed through Prisma 7. Every learning record hangs off a Goal (one subject and exam), so deleting a goal cascades cleanly.

Entity	Purpose and key fields
User	Student account; guest flag; Hinglish preference.
Goal	Subject, exam date, minutes per day, status (draft → diagnosing → active), skipped dates.
Concept · ConceptEdge	Name, unit, weightage (PYQ or estimated), estimated minutes; prerequisite edges between concepts.
Mastery	θ , weighted evidence count and last-practised time — one row per concept.
Question · Attempt	MCQ, numeric or short questions with difficulty and source; every answer with its outcome and misconception.
Session · TutorTurn	Planned vs. actual minutes, mastery before/after and ladder stage; each tutor turn with evaluation and citations.
Mistake	Wrong answer, misconception label, explanation and resolved flag.
MasteryEvent	Every θ change, for the trend chart; seeded history is flagged as demo data.
Resource · Chunk	Uploaded syllabus, notes and PYQs with processing status; page-level text chunks with vector(768) embeddings.

9 Technology Stack

Layer	Choice	Why
Language	TypeScript (strict)	One language front to back; shared Zod schemas.
Framework	Next.js 16 (App Router), React 19	UI, server actions and auth in one deployable.
UI	Tailwind CSS 4, shadcn/ui, custom dot-matrix design system	Fast, consistent components we own.
Visuals	React Flow + dagre; Recharts	Real node-and-edge concept graph; mastery trends.
Validation	Zod 4 + react-hook-form	Same validation on client and server.
Auth	Auth.js v5 — guest demo + Google OAuth (verified email)	Judges skip sign-up; no vendor lock-in.
Database	PostgreSQL + pgvector (Neon; Docker locally)	Relational data and vectors in one database.

Layer	Choice	Why
ORM	Prisma 7 with the pg driver adapter	Typed queries and migrations.
AI	Vercel AI SDK v7 + Gemini (fallback chain); unpdf	Structured output, native PDF input, offline fallback.
Storage	Vercel Blob (optional)	Archive of uploaded PDFs.
Quality	Vitest (177 tests), ESLint, TypeScript, GitHub Actions CI	Engine, AI guard, parsers and retrieval tested on every push.
Hosting	Vercel + Neon	Preview deploy per pull request; zero infrastructure work.

Deliberately not used: a separate backend (two deployables, duplicated types), LangChain (heavy for four prompts and one retrieval), a separate vector database (pgvector is enough at this scale), WebRTC audio rooms (the most fragile thing to demo live) and MongoDB (the data is relational).

10 Trust, Honesty and Security

Three honesty rules are part of the product: the tutor **won't hand you answers** before the worked step; the app **won't invent exam scores** (no readiness percentages; demo data and estimated weightage are labelled); and it **shows its confidence** on every mastery value and recommendation.

A four-lens security audit (authorisation, input abuse, injection, configuration) was run on the whole codebase, with every finding independently re-checked. All confirmed findings were fixed:

Threat	Control now in place
Unlimited guest accounts (database exhaustion)	Per-IP and global sign-up limits inside Auth.js authorize(), so both entry points are covered; guests expire after 7 days; the limiter fails closed.
AI cost abuse (burning the API key)	Postgres-backed per-user limits on tutor, question generation, explanations and uploads; a deployment-wide daily AI budget that falls back to offline mode; regeneration cooldowns; 40-turn session cap.
Prompt injection via notes, papers or chat	Untrusted text only inside nonce-tagged data blocks; history sent as real chat roles; output schema-validated; concept ids, ladder stage and citations recomputed or validated by code.
Cross-account access (IDOR)	Every action validates ids with zod and re-checks ownership; every page runs its own guard; every retrieval query is scoped to the goal.
Malicious uploads / ReDoS	10 MB cap, %PDF magic bytes (MIME ignored), page and text-length caps, sanitised names (incl. bidi characters), parsers bounded (200 KB adversarial input in < 30 ms).
SQL injection	Raw SQL only through Prisma tagged templates; vectors built from validated numbers.
XSS, clickjacking, sniffing	React escaping plus a text-only rich-text renderer; Content-Security-Policy, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy, HSTS; X-Powered-By removed.
Account takeover via Google	Only verified Google emails sign in, and a Google login can never take over a guest row. Secrets stay in git-ignored .env files.

11 Design and Responsiveness

A design review removed the patterns that made the first build look generated: the bright default scrollbar on the black theme (fixed with a per-theme color-scheme and a thin, theme-coloured scrollbar), serif micro-labels, pulsing and blinking decorations, a fake typewriter effect, filler statistics and slang copy. The brand stays: a black

page, one red accent reserved for the next action, and the dot-matrix face kept for the logo and a few key numbers.

- **Phones and tablets:** below 1024 px the navigation becomes a bottom tab bar with safe-area padding; tiles collapse to one column with tighter padding; the concept graph offers a list view and opens details in a sheet; inputs use 16 px text so iOS doesn't zoom.
- **Touch and accessibility:** 44 px touch targets on touch screens, visible focus rings, labelled controls, colour never the only signal, and no motion under reduced-motion.
- **Verified:** every page was rendered at 390, 820 and 1440 px with zero horizontal overflow.

12 Build Status (29 September 2026)

The full loop is implemented and runs end to end, with Gemini and in offline mode:

- **Pages:** landing; sign-in (demo, own syllabus, Google); onboarding (subject → syllabus → editable concept graph → past papers → notes); adaptive diagnostic; dashboard; plan; concept-graph explorer; Socratic learn session with check questions and results; mistake log with Explain My Mistake; revision mode; settings.
- **AI and grounding:** Gemini with a fallback chain, an offline tutor, offline syllabus and paper parsers, verified question generation, and retrieval over the student's notes with pgvector and a BM25 fallback.
- **Demo account:** 27 DBMS concepts, 110 hand-checked questions, 27 tutor scripts, 27 past-paper questions, 32 note pages and ten days of labelled history, seeded in about 0.5 s.
- **Quality:** 177 unit tests passing, clean lint and typecheck, a successful production build, and GitHub Actions CI. Browser runs of the demo, a learn session and the full new-syllabus journey pass without console errors.
- **Before launch:** deploy on Vercel with a Neon database dedicated to SyllabusOS, add the production Google redirect URI, and rotate the Google client secret.

13 Roadmap

- **Voice Socratic mode** — speak your reasoning; the tutor replies by voice using browser speech APIs (no WebRTC server).
- **Class heatmap for teachers** — aggregate misconceptions to show where a class is stuck, not just final grades.
- **Hinglish and regional languages** — explanations in the student's language with technical terms kept in English.
- **Study rooms** — shared progress for accountability, without sharing private data.
- **Placement mode** — job description → skills graph → the same loop.

14 Conclusion

Built by Team Crazy Coders in four lanes (AI pipelines, core engine and backend, frontend, integration and demo) with issue-per-feature branches, reviewed pull requests and CI on every push.

Students already have content, deadlines and chatbots that will do their homework for them; none of that makes them pass. SyllabusOS builds the missing layer: it decides what to learn next, explains why, and checks that it was actually learned. Decisions come from a transparent, deterministic engine; teaching is Socratic and grounded in the student's own notes; and every answer feeds back into the plan.

We don't answer students' questions — we decide the next best thing for them to learn, explain why, and make sure they actually learn it.